

Vaishnavi Badoge

AIM : Data Visualization with Matplotlib: Data visualization on a dataset using matplotlib and seaborn libraries

Scafler Plot

Line Plot

Histogram

Box Plot

Pair Plot

Theory :

### 1. Importance of Data Visualization in Data Analysis & EDA

Ans: Data visualization is an essential part of data analysis because it converts complex numerical information into clear and easy-to-understand visual forms. It enables analysts to quickly recognize patterns, trends, and relationships that may not be visible in raw data. In Exploratory Data Analysis (EDA), visualization is particularly valuable as it helps identify outliers, analyze data distributions, and reveal hidden insights before applying machine learning techniques. Visual tools such as scatter plots, histograms, and box plots make it easier to explore data effectively and support better decision-making.

### 2. Difference Between Matplotlib and Other Visualization Libraries

Ans: Matplotlib is a core plotting library in Python that offers detailed control over how visualizations are created. It supports a wide range of static, animated, and interactive plots. Compared to libraries like Seaborn and Plotly, Matplotlib operates at a lower level, which means it often requires more code but allows greater customization. Seaborn is built on top of Matplotlib and provides simpler, high-level functions along with better default styling for statistical graphics. In contrast, Plotly is designed for creating interactive visuals, especially for web-based applications. Therefore, Matplotlib is ideal for precise customization, while Seaborn and Plotly focus more on simplicity and visual appeal.

### 3. Purpose of the Pyplot Module in Matplotlib

Ans: The pyplot module in Matplotlib offers a simple and user-friendly way to create and manage plots. It follows a MATLAB-style approach, allowing users to easily generate figures, draw plots, label axes, and display results using basic commands. Pyplot takes care of common tasks such as creating figures, plotting lines, adding titles, and enabling grids, which makes it especially helpful for beginners. In short, it serves as the primary interface for most plotting activities in Matplotlib without requiring users to work with more complex object-oriented methods.

### 4. Process of Creating a Basic Line Plot

Ans: Creating a simple line plot in Matplotlib involves a few clear steps. First, the necessary library (matplotlib.pyplot) is imported. Next, the data for the x-axis and y-axis is defined. The plot() function is then used to draw the line graph. After that, axis labels and a title are added to make the visualization more meaningful. A grid can also be included to improve readability. Finally, the plot is displayed using the show() function. This straightforward process helps users easily visualize patterns or changes in data over time.

### 5. Importance of Customization in Data Visualization

Ans: Customization plays a key role in data visualization because it enhances clarity, readability, and overall presentation. Without proper adjustments, graphs can appear unclear and may not effectively convey the intended message. Matplotlib offers a wide range of customization options, including modifying colors, line styles, markers, labels, titles, font sizes, legends, and grid styles. Additionally, users can adjust figure size, set axis limits, and add annotations to emphasize important data points. These capabilities allow the creation of clear, attractive, and informative visualizations suited to specific requirements.

Import Libraries

Create Sample Dataset

Scafler Plot

Line Plot

Bar Plot

Histogram

Box Plot

Pair Plot

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import seaborn as sns

# Creating a sample dataset
np.random.seed(42)

data = pd.DataFrame({
    'X': np.arange(1, 101),
    'Y': np.random.randint(10, 100, 100),
    'Category': np.random.choice(['A', 'B', 'C'], 100)
})

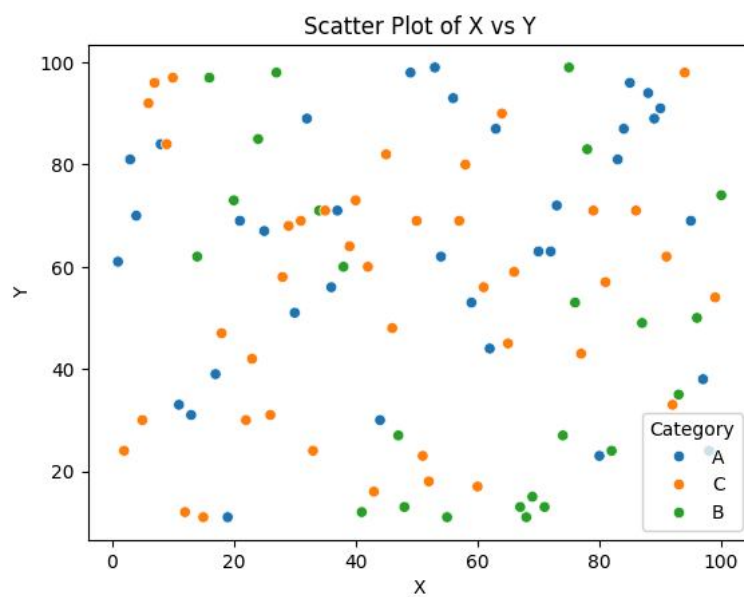
data.head()
```

|   | X | Y  | Category |
|---|---|----|----------|
| 0 | 1 | 61 | A        |
| 1 | 2 | 24 | C        |
| 2 | 3 | 81 | A        |
| 3 | 4 | 70 | A        |
| 4 | 5 | 30 | C        |

Next steps: [Generate code with data](#) [New interactive sheet](#)

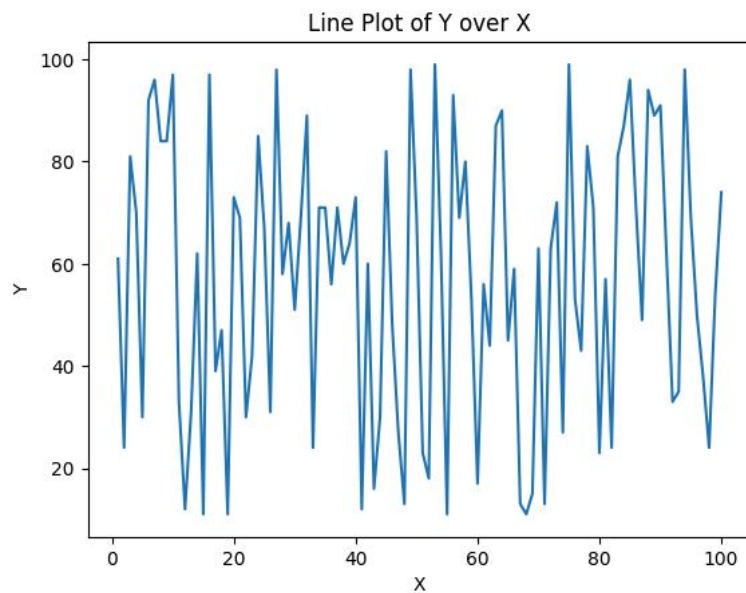
### Scatter Plot

```
sns.scatterplot(data=data, x='X', y='Y', hue='Category')
plt.title('Scatter Plot of X vs Y')
plt.show()
```



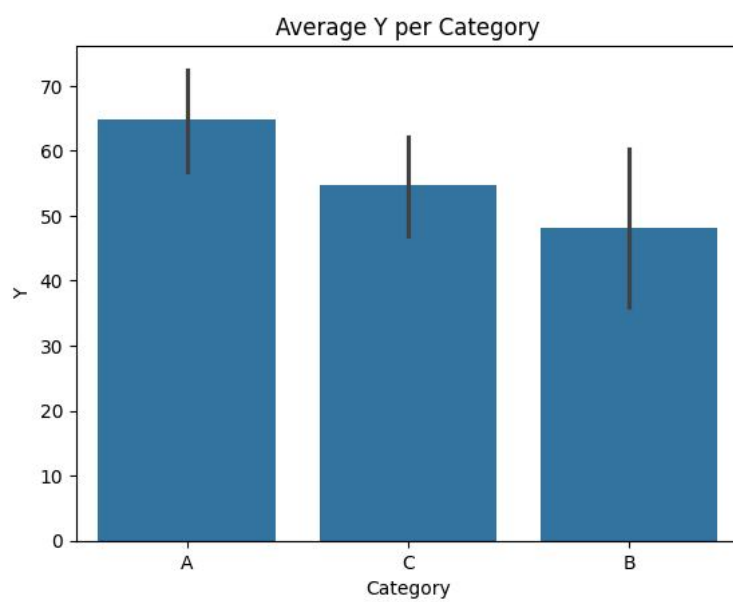
### Line Plot

```
sns.lineplot(data=data, x='X', y='Y')
plt.title('Line Plot of Y over X')
plt.show()
```



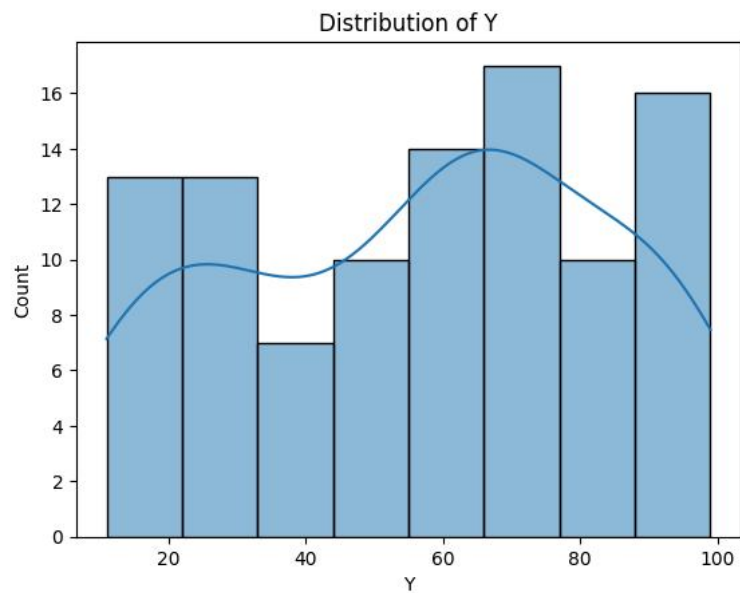
### Bar Plot

```
sns.barplot(data=data, x='Category', y='Y')  
plt.title('Average Y per Category')  
plt.show()
```



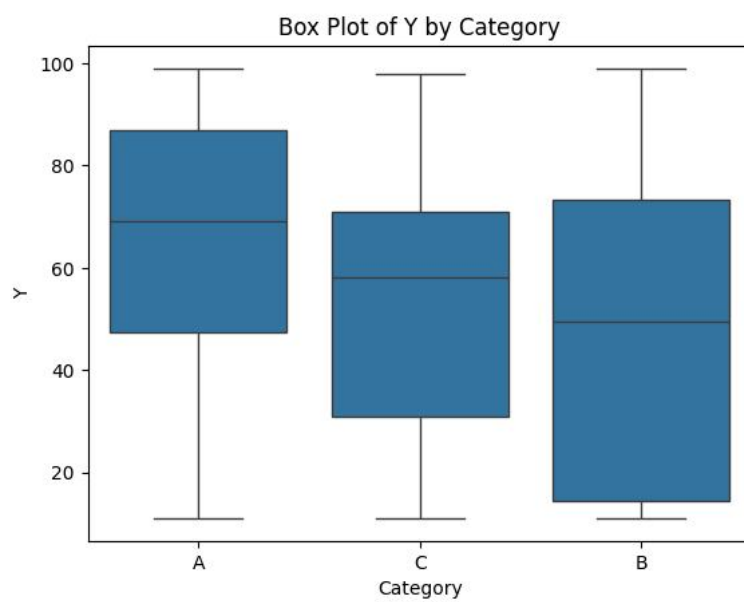
### Histogram

```
sns.histplot(data=data, x='Y', kde=True)  
plt.title('Distribution of Y')  
plt.show()
```



### Box plot

```
sns.boxplot(data=data, x='Category', y='Y')  
plt.title('Box Plot of Y by Category')  
plt.show()
```



### Pair Plot

```
sns.pairplot(data, hue='Category')  
plt.show()
```

